

Lelethu Bala

PHASE 1: SQL DATA EXPLORATION Focus: Aggregations, Grouping, Filtering

2 days ago (2s)

3

SQL

```
%sql
-- Displaying all the records of the dataset
SELECT * FROM workspace.dataforge.dfa_insurance_project_raw_dataset;
```

See performance (1)

Optimize

Table

	Customer_ID	Age	Gender	Province	Monthly_Income	Join_Date	Policy_ID	Policy_Type	Prer
1	CUST1000	44	Male	Gauteng	26064	2020-05-30	POL5000	Home	1495
2	CUST1001	41	Male	Free State	3500.0	2025-06-16	POL5001	Auto	746.28
3	CUST1002	33	Female	Limpopo	29552.92	2020-06-14	POL5002	Health	2325.42
4	CUST1003	41	Female	western cape	3500.0	2024-06-10	POL5003	Auto	885.13
5	CUST1004	42	Female	Mpumalanga	3500.0	2024-07-15	POL5004	Home	R 1,875.
6	CUST1005	33	Female	Limpopo	49905.01	2024-04-13	POL5005	Home	2210.31
7	CUST1006	38	Male	Free State	49905.01	2022-12-19	POL5006	Motor	433.09
8	CUST1007	30	FEMALE	KwaZulu-Natal	20975.4	10-14-2021	POL5007	Life	817.56
9	CUST1008	42	Male	North West	31654.77	2022-03-28	POL5008	Home	R 1,410.
10									

550 rows | 2.18s runtime

Refreshed 2 days ago

The total number of rows of the untouched raw dataset

2 days ago (3s)

```
%sql
-- Counting total rows:
SELECT COUNT(*)
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
```

See performance (1)

Table

	COUNT(*)
1	550

1 row | 2.85s runtime

1. Total number of customers

2 days ago (2s)

```
%sql
-- 1. Total number of customers
SELECT COUNT (DISTINCT Customer_ID ) AS total_customers
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
```

See performance (1)

Table

	total_customers
1	518

1 row | 1.74s runtime

2. Total number of policies

2 days ago (2s)

```
%sql
-- 2.Total number of policies
SELECT COUNT (DISTINCT Policy_ID ) AS total_policies
FROM workspace.dataforge.dfa_insurance_project_raw_dataset;
```

> [See performance \(1\)](#)

	1.2 total_policies
1	520

1 row | 1.77s runtime

3. Total number of claims

2 days ago (2s)

```
%sql
-- 3.Total number of claims
SELECT COUNT ( Claim_ID ) AS total_claims
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
```

> [See performance \(1\)](#)

	1.2 total_claims
1	547

1 row | 2.41s runtime

4. Total claim amount

2 days ago (2s) 9

```
%sql
-- 4. Total claim amount
SELECT SUM(CAST(REGEXP_REPLACE(Claim_Amount, '[R, ]', '') AS DOUBLE)) AS total_claim_amount
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
```

> [See performance \(1\)](#) [Optimize](#)

	1.2 total_claim_amount
1	12338859.64

1 row | 1.55s runtime

5. Claims by policy type

2 days ago (2s)

```
%sql
--5.Claims by policy type
SELECT Policy_Type, COUNT(Claim_ID) AS total_claims
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
GROUP BY Policy_Type
ORDER BY total_claims DESC
```

> [See performance \(1\)](#)

	Policy_Type	total_claims
1	Health	124
2	Auto	115
3	Life	96
4	Home	83
5	Funeral	63
6	Motor	10
7	AUTO	8
8	home	7
9	LIFE	7
10	health	6
11	auto	5
12	Medical	5
13	HOME	4
14	HEALTH	4
15	funeral	3

18 rows | 2.07s runtime

6. Average claim amount per policy type

```
--6.Average claim amount per policy type
SELECT Policy_Type, AVG(CAST(REGEXP_REPLACE(Claim_Amount, '[R, ]', '') AS DOUBLE)) AS avg_claim_amount
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
GROUP BY Policy_Type
ORDER BY avg_claim_amount DESC
```

> [See performance \(1\)](#) [Optimize](#)

	Policy_Type	avg_claim_amount
1	life	54691.755000000005
2	Life	46282.957872340434
3	LIFE	41811.555
4	HOME	28320.537500000002
5	Home	25697.625180722887
6	auto	23677.469999999998
7	home	21122.07714285714
8	Property	20201.643333333333
9	Auto	18950.71318584071
10	HEALTH	18886.285
11	Medical	17714.302
12	FUNERAL	16378.670000000002
13	AUTO	15657.644999999999
14	funeral	15281.15
15	Motor	14929.177

18 rows | 46.27s runtime

7. Number of fraud vs non-fraud claims

12:14 AM (2s)

```
%sql
--7.Number of fraud vs non-fraud claims
SELECT Fraud_Flag,COUNT(Fraud_Flag) AS total_claims
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
GROUP BY Fraud_Flag
```

> [See performance \(1\)](#)

	^B _C Fraud_Flag	¹ ₂ total_claims
1	No	395
2	0	18
3	Yes	73
4	NO	18
5	null	0
6	N	20
7	no	10
8	1	3
9	YES	3

9 rows | 1.86s runtime

8. Top 5 highest claim amounts

Just now (12s) 13

```
%sql
-- 8.Top 5 highest claim amounts
SELECT CAST(REGEXP_REPLACE(Claim_Amount, '[R, ]', '') AS DOUBLE) as Claim_Amounts
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
ORDER BY Claim_Amount DESC
LIMIT 5;
```

> [See performance \(1\)](#) [Optimize](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [Claim_Amounts: double]`

	¹ ₂ Claim_Amounts
1	64294.74
2	62922.32
3	58664.79
4	53437.94
5	53164.54

5 rows | 11.83s runtime

9. Claims per location

```
-- 9. Claims per location
SELECT Province, COUNT(Claim_ID) AS total_claims
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
GROUP BY Province
ORDER BY total_claims DESC
```

> [See performance \(1\)](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [Province: string, total_claims: long]`

	Province	total_claims
1	North West	62
2	Eastern Cape	62
3	KwaZulu-Natal	62
4	Free State	61
5	Western Cape	56
6	Gauteng	55
7	Mpumalanga	48
8	Limpopo	41
9	null	13
10	WC	10
11	EC	8
12	western cape	8
13	kwazulu-natal	6
14	north west	6
15	FS	6

27 rows | 1.65s runtime

10. Claims trend over time

```
-- 10. Claims trend over time .
SELECT Claim_Date AS date, COUNT(Claim_ID) AS total_claims
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
GROUP BY date
ORDER BY date ASC
```

> [See performance \(1\)](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [date: string, total_claims: long]`

	date	total_claims
27	08-13-2025	1
28	08-22-2025	1
29	08/01/2024	1
30	08/03/2025	1
31	09-20-2025	1
32	09-Apr-2024	1
33	09-Jul-2024	1
34	09-Jul-2025	1
35	10-06-2024	1
36	10-10-2024	1
37	10-11-2025	1
38	10-28-2025	2
39	11-10-2024	1
40	11-11-2024	1
41	11-23-2024	1

436 rows | 2.19s runtime